

SIMATS ENGINEERING

Department of Computer Science & Engineering ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Computer Architecture

Course Code: CO Assessed: Weightage:
CSA1223 C02

20%
Course Title: Assessment:

ASSESSMENT TOOLG:
DEBUGGING &
OPTIMISATION TASK 100

Total Marks:

CO2 Statement: Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)

Student Name: R.Ahgen Abiheh ANNEXURE A (4231/o34

Department:

|4-0R-26

Reg.No.: Date:

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look ahead adder, and explain how it reduces delay.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors.

Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

R Ahaen hbihele

(Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this

assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

Identification of Errors / Bugs(20%)	Q3	Correctness of Debugged Solution(25%)
	Q4	Optimization Techniques Applied(20%)
Q1	Q5	Testing and Documentation(15 %)
Q2	Debugging Approach & Analysis(20%)	

2

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20% 25%	Applies suitable Uses effective debugging techniques with clear analysis and root cause identification(4)	debugging methods with minor gaps(3)	No systematic methods with limited analysis(2)	debugging approach followed(0)
Correctness of Debugged Solution		Provides completely corrected solution with with minor errors(4)	Provides working solution	Partially correct solution with	Incorrect solution or fails to resolve
Jaccurate functionality and expected output(5)	Performs limited functionality(3)	Limited errors(0)	No optimization Optimization memory usage and quality(4)	Techniques Applied 20% optimization methods to improve performance, implementation(0)	optimization optimization with) with noticeable minimal applied or inefficient Provides
Applies efficient		improvements(3) improvements(2)			
Testing and Documentation	15%	testing adequate testing Limited testing and clear documentation(3)	documentation	No proper testing or with proper validation and	and incomplete documentation(1) with minor documentation provided(0(
					issues(2)

SIMATS ENGINEERING — Department of Computer Science & Engineering

Assessment Tool 2 — Debugging & Optimisation Task | CSA1223 Computer Architecture | CO2

Assessment Tool 2 — Debugging & Optimisation Task

Course Code: CSA1223 **Course Title:** Computer Architecture **CO Assessed:** CO2
Weightage: 20% **Total Marks:** 100

Question 1

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Debug Analysis

Given data: X = +120, Y = +50, 8-bit signed 2's complement, valid range -128 to +127. **Step 1 — Represent the operands**

120 = 64+32+16+8 → **+120 = 01111000**. 50 = 32+16+2 → **+50 =**

00110010. **Step 2 — Perform the addition exactly as the buggy program does**

```
01111000 (+120)
+ 00110010 (+50)
-----
10101010
```

Step 3 — Interpret the raw bit pattern

The MSB of 10101010 is 1, so the hardware/program reads this as a negative number. Taking the 2's complement to find its magnitude: invert → 01010101, add 1 → 01010110 = 64+16+4+2 = 86. So the program reports the sum as **-86**.

Step 4 — Locate the bug

The true mathematical sum is 120 + 50 = **+170**, but the largest value an 8-bit signed field can hold is +127. Since 170 > 127, the result cannot fit — this is a classic **overflow**, and the program is bugged because it is not checking for this condition before trusting the raw bit pattern; it silently wraps around to -86 and presents it as a valid answer.

Step 5 — Confirm using the sign rule for overflow

Overflow rule for addition: if two operands of the **same** sign produce a result of the **opposite** sign, overflow has occurred. Here both operands are positive but the result's sign bit is 1 (negative) → **Overflow = YES**, which the buggy program failed to check.

Step 6 — Hardware-level confirmation

The carry-based test gives the same diagnosis: V = C_in(into MSB) XOR C_out(out of MSB). Adding bit 6 column (1+1=10, carry 1 into bit 7) while bit 7 column (1+0+carry1 = 0, carry 0 out) gives C_in=1, C_out=0, so V = 1 XOR 0 = **1 (overflow set)**.

Correction / Proposed Fix

1. **Always evaluate the overflow flag, not just the bit pattern.** After every signed addition, compute V = C_in C_out (or equivalently check “same-sign operands,

⊕ opposite-sign result”) and raise an exception / set an error flag whenever $V = 1$, instead of blindly accepting the wrapped-around value.

2. **Widen the register before adding**, when the application can tolerate it — performing the addition in 16-bit or 32-bit arithmetic ($120 + 50 = 170$, which fits comfortably) avoids the overflow altogether and is the simplest fix at the software level.
3. **Use saturating arithmetic** where appropriate (common in DSP/embedded code): if overflow is detected, clamp the result to +127 instead of letting it wrap to a nonsensical negative value.

Final Answer

10101010 (interpreted as -86) is **incorrect** because signed overflow occurred; the true sum 170 exceeds the 8-bit signed maximum of 127. With overflow detection ($V=1$) in place, the program should reject/flag this result rather than return -86.

Interpretation

This example demonstrates why overflow detection cannot be skipped in signed arithmetic — the ALU will always produce *some* 8-bit pattern, but that pattern is only meaningful if it stays inside the representable range. Debugging this class of bug means checking the overflow flag on every arithmetic instruction that touches signed values close to the register’s limits, not just trusting the numeric output.

Question 2

A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

Debug Analysis

Given data: A = 1001 (9), B = 0111 (7), 4-bit adder.

Step 1 — Trace the ripple carry adder’s critical path

In a ripple carry adder built from 4 cascaded full adders (FA0–FA3), FA_i cannot compute its carry-out until FA_(i-1) has produced a valid carry-out, because $Cout_i = Gi + Pi \cdot Cin_i$ and Cin_i is literally the previous stage’s output. So the carry has to “ripple” physically through every stage before the final sum settles:

FA0 → C1 → FA1 → C2 → FA2 → C3 → FA3 → C4

For n bits, worst-case delay $\approx n \times t_{\text{carry}}$ (about 2 gate delays per full adder stage), so a 4-bit RCA already costs ≈ 8 gate delays, and this **grows linearly** — a 32-bit RCA would need roughly 8× the delay of the 4-bit version. That linear growth is the root cause of the bottleneck in the real-time system: as word width increases (32-bit, 64-bit ALUs), the ripple delay becomes the critical path of the whole processor clock cycle.

Step 2 — Root cause confirmed

The bottleneck is *sequential dependency between adjacent full-adder stages* — every stage must wait for its neighbor, so parallel hardware capacity is wasted.

Optimized Solution: Carry Look-Ahead Adder (CLA)

Step 3 — Compute Generate/Propagate for the same inputs

$$G_i = A_i \cdot B_i, P_i = A_i \oplus B_i$$

i A_i B_i G_i P_i

0 1 1 1 0

1 0 1 0 1

2 0 1 0 1

3 1 0 0 1

Step 4 — Carries computed directly (not chained)

$$C_1 = G_0 + P_0 C_0 = 1$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 0 + (1)(1) + 0 = 1 \quad C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 = 0 + 0 + (1)(1)(1) + 0 = 1$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + \dots = 0 + 0 + (1)(1)(0) + (1)(1)(1)(1) + 0 = 1$$

Sum: $S_0 = P_0 C_0 = 0$, $S_1 = P_1 C_1 = 0$, $S_2 = P_2 C_2 = 0$, $S_3 = P_3 C_3 = 0$, with final carry $C_4 = 1$, $\oplus \oplus \oplus \oplus$ giving **10000 (16)**.

Verification: $9 + 7 = 16$. ✓

Step 5 — Why this fixes the delay bottleneck

Every C_i above is expressed only in terms of $G_0..G_i$, $P_0..P_i$ and C_0 — quantities that are all available at the *same* time (immediately after the first-level AND/XOR gates that compute the individual G_i/P_i). A two-level AND–OR network then produces every carry **simultaneously**, so

the adder no longer waits for a physical ripple. Delay becomes roughly constant (≈ 4 gate delays) regardless of the bit-width.

Metric Ripple Carry Adder Carry Look-Ahead Adder Delay for n bits $\approx n \times t_{\text{carry}}$
(linear) $\approx$ constant, independent of n

Suitability for real-time systems

Degrades as width grows Maintains a predictable, low latency

Final Answer

Replacing the RCA with a 4-bit CLA gives the same correct sum ($1001+0111=10000=16$) but removes the linear carry-propagation delay, which is the direct cause of the real-time system's slowdown.

Interpretation

The debugging insight here is that the *functional* output of the ripple carry adder was correct — the bug was purely a **timing** defect caused by sequential carry dependency. Because CLA hardware pre-computes all carries from independent generate/propagate terms, it directly optimizes the critical path length, which is exactly the metric that matters for a real-time system's clock speed.

Question 3

A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

Debug Analysis

Given data (test case used to expose the bug): Multiplicand $M = -5$, Multiplier $Q = -4$ — both negative, a case that commonly exposes Booth implementation bugs.

Step 1 — Correct 5-bit 2's complement representation (needed to catch the bug)

$+5 = 00101 \rightarrow \text{invert} \rightarrow 11010 \rightarrow +1 \rightarrow \mathbf{M = 11011 (-5)}$. $+4 = 00100 \rightarrow \text{invert} \rightarrow 11011 \rightarrow +1 \rightarrow \mathbf{Q = 11100 (-4)}$.

Step 2 — Likely sources of error in a faulty implementation

A Booth multiplier commonly misbehaves on negative operands for one of these reasons:

1. **Logical shift instead of arithmetic shift** — if the right shift of (A, Q, Q_{-1}) does not $\neg$ replicate the sign bit of A , a negative partial product loses its sign after every shift, corrupting the rest of the computation.
2. **Wrong bit-pair table** — treating “11” as an operation (instead of “no operation”) or swapping the “10”/“01” rules reverses the direction of the correction, which only shows up when the multiplier itself has runs of 1s (as $-4 = 11100$ does).
3. **Missing sign extension of M** when M is loaded into a register wider than its minimum width, so that $A - M / A + M$ is computed against a corrupted multiplicand.

The example below re-runs the algorithm carefully, applying the correct rules, to show what the *correct* trace looks like and to isolate where a faulty implementation would diverge.

Step 3 — Correct rule table (for reference)

$Q_0 Q_{-1} = 00$ or $11 \rightarrow$ no operation, just arithmetic shift; $10 \rightarrow A = A - M$; $01 \rightarrow A = A + M$. $\neg$

Step 4 — Corrected iteration trace

Initial: $A = 00000$, $Q = 11100$, $Q_{-1} = 0$. $\neg$

Iter	$Q_0 Q_{-1}$	Operation	A after op	$\rightarrow A, Q, Q_{-1}$	Arithmetic shift
1	0 0	no-op	00000	00000, 01110, 0	
2	0 0	no-op	00000	00000, 00111, 0	
3	1 0	$A = A - M$	00101	00010, 10011, 1	
4	1 1	no-op	00001	01001, 1	
5	1 1	no-op	00000	10100, 1	

(Every shift preserves the sign bit of A — this **arithmetic** shift, as opposed to a logical shift, is the step where a buggy implementation most often diverges for negative operands.)

Step 5 — Final product

After 5 iterations, A = 00000, Q = 10100. Concatenated, A:Q = 0000010100 = $_2 +20$ (the leading bit is 0, so this reads directly as positive).

Verification: $(-5) \times (-4) = +20$. ✓

Correction Summary

Bug source Symptom Fix

Logical shift used instead of arithmetic shift	each shift; wrong magnitude for negative products	Always sign-extend A during the (A,Q,Q) right shift $\rightarrow$
--	---	---

Sign of A corrupted after		
Incorrect Q0Q decode table $\rightarrow$ Extra or missing add/subtract, especially with multipliers containing 1-runs	Re-verify the four cases (00, 01, 10, 11) against the standard table before coding	

M not correctly sign extended	Wrong $A \pm M$ outcome even when the shift logic is right	complement with enough bits to cover the full product width before the first
	Represent M in 2's	

Bug source Symptom Fix

iteration

Final Answer

Corrected product of $(-5) \times (-4) = 0000010100 = +20$ (decimal) $_2$.

Interpretation

Booth's algorithm is only self-correcting for sign as long as two hardware details are respected exactly: the shift must be **arithmetic** (sign-preserving), and the multiplicand must be a properly sign-extended 2's complement value before any $A \pm M$ step.

Debugging a Booth multiplier that fails specifically on negative operands should start by checking those two details first, since they are the most common points where a naive re-implementation diverges from the correct algorithm.

Question 4

An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non restoring division method, explaining how it improves performance.

Debug Analysis

Given data: Dividend = 11 (1011), Divisor = 2 (0010), 4-bit unsigned

division. **Step 1 — Trace the restoring algorithm and flag every**

wasted cycle Initialise A = 00000, Q = 1011, M = 00010.

Wasted

Cycle Shift $\rightarrow$ A,Q A - M Sign restore? Q0

1 00001, 0110
 11111 Neg Yes —
 0
restore A
= A+M
 2 00010, 1100
 00000 Pos No 1

11111 Neg Yes —
 0
restore A
= A+M
 4 00011, 0100
 00001 Pos No 1

3 00001, 1010

Result: $Q = 0101 = 5$, A (remainder) = 00001 = 1. (Check: $2 \times 5 = 10$,

$11 - 10 = 1$.) ✓ **Step 2 — Identify the inefficiency**

Cycles 1 and 3 each perform a subtraction that turns out to be wrong (the trial quotient bit would have been 0), so the algorithm must immediately execute a **second** arithmetic operation — adding the divisor back — purely to undo the mistake. That is 2 extra additions on top of the 4 required subtractions, i.e. **6 arithmetic operations total for a 4-bit division**. In a real-time embedded loop running many divisions per second, these “throwaway” restore operations directly inflate execution time without producing any additional information.

Optimized Solution: Non-Restoring Division

Step 3 — Rule: if the previous remainder was ≥ 0 , shift and subtract M ; if it was negative, shift and add M instead — the “wrong guess” is corrected by the *next* cycle’s operation choice, so no dedicated restore step is ever needed mid-loop.

Cycle Operation A after op Sign Q_0

1 shift, $A-M$ 11111 Neg 0
 2 (prev neg) shift, $A+M$ 00001 Pos 1
 3 (prev pos) shift, $A-M$
 4 (prev neg) shift, $A+M$
 00000 Pos 1 11111 Neg 0

Final A is non-negative, so **no end correction is even needed** this time. Result: $Q = 0101 = 5$, remainder $A = 00001 = 1$ — identical to the restoring result.

Step 4 — Count the operations

Non-restoring division performed exactly **4 arithmetic operations** (one add-or-subtract per cycle, no separate restores), compared with **6** for the restoring method on the same inputs — a real reduction in execution time for the embedded system.

Final Answer

$11 \div 2 \rightarrow$ **Quotient = 5, Remainder = 1** for both methods, but non-restoring division reaches it in 4 operations instead of 6.

Interpretation

The bug/inefficiency was never in the *correctness* of restoring division — both methods agree on the answer — it is in the **workflow**: every negative trial subtraction forces a redundant “undo” addition. Non-restoring division optimizes this by folding the correction into the next cycle’s choice of operation, cutting the number of arithmetic operations roughly in half in the worst case, which is exactly the kind of fixed-operation-count, branch-light behaviour that matters for predictable timing in an embedded system.

Question 5

A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Debug Analysis

Given data (test case that exposes the bug): Large = $2^3 = 1,073,741,824$, Small = 1 (i.e. 2^0), both being added in IEEE 754 single precision (1 sign + 8 exponent + 23 mantissa bits).

Step 1 — Normalize both operands

Large = 1.0×2^3 . Small = 1.0×2^0

Step 2 — Align exponents before adding

The addition hardware/software must shift the smaller-exponent mantissa right until its exponent matches the larger one. Here the exponent difference is $30 - 0 = 30$ places.

Step 3 — Compare the shift amount with the mantissa width

Single precision only stores 23 explicit mantissa bits. Shifting Small’s mantissa right by 30 positions pushes every one of its significant bits **past** the last available mantissa bit position — there is no room left to keep any of them.

Step 4 — Observe the (buggy-looking) result

Large + Small (single precision) = Large (unchanged)

The program reports $1,073,741,824 + 1 = 1,073,741,824$ — Small appears to have vanished entirely. This is not a coding bug in the arithmetic unit; it is the expected, documented consequence of IEEE 754’s fixed 23-bit mantissa once the exponent gap exceeds the mantissa width. Debugging this correctly means recognising that the “incorrect result” is a **representational precision limit**, not a logic error, so the fix has to change the representation or the algorithm — not the addition code itself.

Step 5 — Where partial (rather than total) precision loss occurs

For a smaller exponent gap — say 20 instead of 30 — some, but not all, of Small’s bits would survive alignment, and the final sum would then need **rounding** (round-to-nearest-even, by default) to fit the result back into 23 mantissa bits. This intermediate case is the more common source of subtly “slightly wrong” scientific results, as opposed to the complete disappearance seen when the gap exceeds 23.

Optimization Techniques

1. **Switch to double precision.** With 52 mantissa bits instead of 23, double precision tolerates an exponent gap almost 30 bits wider before a small operand disappears completely, which resolves this specific failure case (gap of $30 < 52$) and greatly reduces rounding error for more general cases.
2. **Reorder the summation (algorithmic fix).** When many numbers of very different magnitudes must be summed, accumulating them **from smallest to largest** keeps intermediate exponent gaps smaller at each step, preserving more precision than adding a tiny value directly into an already-huge running total.
3. **Compensated (Kahan) summation.** This technique carries a running “lost-bits” correction term alongside the sum and re-injects it on the next addition, which keeps the effective error bounded even when many small values are repeatedly added to a large accumulator — a standard optimization in scientific computing libraries.

Final Answer

Single-precision addition of 2^3 and 2 loses the smaller operand completely because the 2^3 required 30-bit alignment shift exceeds the 23-bit mantissa; switching to double precision (52-bit mantissa) or applying Kahan/ordered summation resolves the accuracy loss.

Interpretation

This debugging exercise highlights a limitation that is intrinsic to floating-point formats rather than a mistake in any particular implementation: **exponent alignment** during addition can silently discard the significant bits of the smaller-magnitude operand whenever the exponent difference exceeds the mantissa width, and the resulting **rounding** is what scientific applications must account for by choosing an appropriately wide format or a numerically stable summation algorithm.